Rapport TP : Design Patterns (Factory & Factory Method)

11 novembre 2026

Table des matières

1 Introduction **3**

2 Diagrammes UML **3**
2.1 1. Méthode Simple Factory 3

3 Implémentation Java **3**
3.1 Capture de l'environnement de développement 3
3.2 2. Design Pattern Factory Method Modifié 5

4 Implémentation Java **6**
4.1 Capture de l'environnement de développement 6

5 Diagrammes UML **9**
5.1 1. Méthode Factory 9

6 Implémentation Java **9**
6.1 Capture de l'environnement de développement 9
6.2 2. Design Pattern Factory Modifié 12

1 Introduction

Ce document présente la mise en œuvre du design pattern **Factory** ainsi que sa variante **Factory Method**.

2 Diagrammes UML

2.1 1. Méthode Simple Factory

Voici le diagramme UML initial de la fabrique simple :

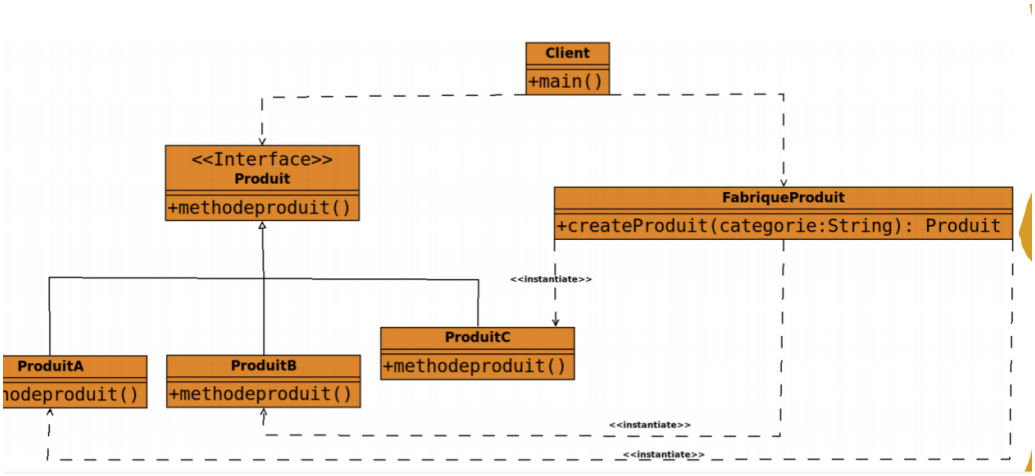


FIGURE 1 – Diagramme UML - Factory method

3 Implémentation Java

3.1 Capture de l'environnement de développement

L'image ci-dessous montre la structure du projet sous IntelliJ IDEA :

Listing 1 – Classe Produit

```
1
2 public abstract class Produit {
3
4     public abstract void methodeproduit();
5
6 }
```

Listing 2 – Classe FabriqueProduit

```
1
2 public class FabriqueProduit {
3
4     public static final String typeA = "A";
5     public static final String typeB = "B";
6     public static final String typeC = "C";
7
8     public Produit createProduit(String type) {
9
```

```
        Produit produit = null;

        switch (type) {

            case typeA:
                produit = new ProduitA();
                break;

            case typeB:
                produit = new ProduitB();
                break;

            case typeC:
                produit = new ProduitC();
                break;

            default:
                throw new IllegalArgumentException(
                        "Type de produit inconnu"
                );
        }

        return produit;
    }

}
```

Listing 3 – Classe ProduitA

```

public class ProduitA extends Produit {

    public ProduitA() {

    }

    public void methodeproduit() {
        System.out.println("je suis un produit de type A");
        System.out.println("produitA.methodeproduit()");
    }

}
```

Listing 4 – Classe ProduitB

```

public class ProduitB extends Produit {

    public ProduitB() {

    }

    public void methodeproduit() {
        System.out.println("je suis un produit de type B");
        System.out.println("produitB.methodeproduit()");
    }

}
```

Listing 5 – Classe ProduitC

```
1
2  public class ProduitC extends Produit {
3
4      public ProduitC() {
5
6      }
7
8      public void methodeproduit() {
9          System.out.println("je␣suis␣un␣produit␣de␣type␣C");
10         System.out.println("produitC.methodeproduit()");
11     }
12
13 }
```

Listing 6 – Classe Client

```
1
2  public class Client {
3
4      public static void main(String[] args) {
5
6          System.out.println("bonjour␣le␣monde");
7
8          FabriqueProduit fabriqueproduit = new FabriqueProduit();
9
10         Produit produit = null;
11
12         // Affiche un produit de type A
13         produit = fabriqueproduit.createProduit(fabriqueproduit.typeA);
14         produit.methodeproduit();
15
16         // Affiche un produit de type B
17         produit = fabriqueproduit.createProduit(fabriqueproduit.typeB);
18         produit.methodeproduit();
19
20         // Affiche un produit de type C
21         produit = fabriqueproduit.createProduit(fabriqueproduit.typeC);
22         produit.methodeproduit();
23
24         // Produit inexistant
25         String d = "D";
26
27         produit = fabriqueproduit.createProduit(d);
28         produit.methodeproduit();
29
30     }
31
32 }
```

3.2 2. Design Pattern Factory Method Modifié

Après restructuration, nous obtenons le diagramme UML suivant avec la méthode de fabrique abstraite :

Repository GitHub : `https://github.com/Allaminehafiz/designPattern.git`[5]

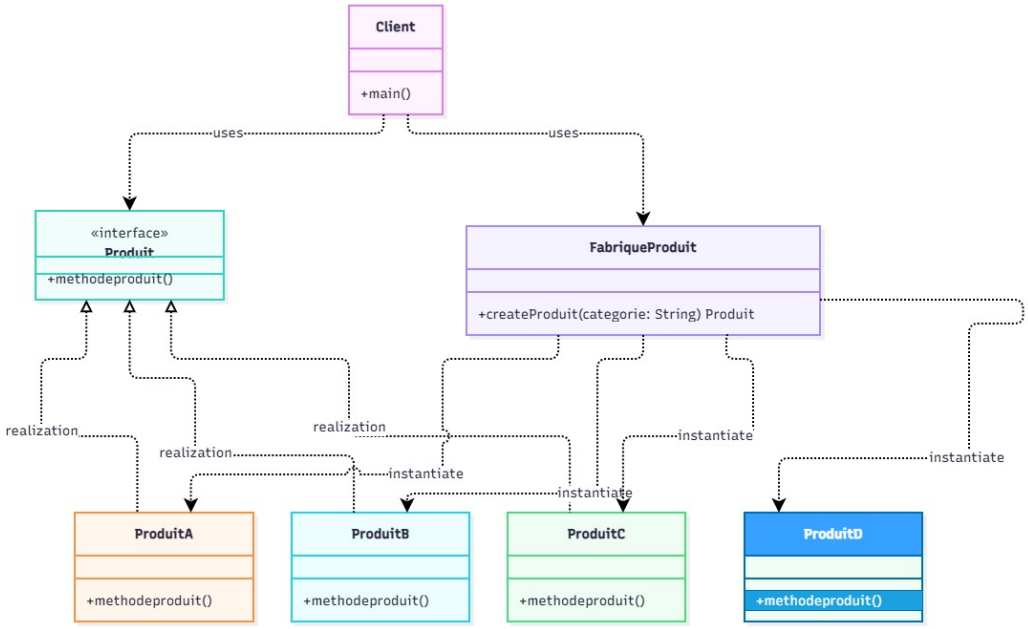


FIGURE 2 – Diagramme UML - Factory Method Modifié

4 Implémentation Java

4.1 Capture de l'environnement de développement

L'image ci-dessous montre la structure du projet sous IntelliJ IDEA ainsi que la classe `ProduitFactoryModifié` :

Listing 7 – Classe Produit

```
1
2 public abstract class Produit {
3
4     public abstract void methodeproduit();
5
6 }
```

Listing 8 – Classe FabriqueProduit

```
1
2 public class FabriqueProduit {
3
4     public static final String typeA = "A";
5     public static final String typeB = "B";
6     public static final String typeC = "C";
7     public static final String typeC = "D";
8
9     public Produit createProduit(String type) {
10
11         Produit produit = null;
12
13         switch (type) {
14
15             case typeA:
16                 produit = new ProduitA();
```

```
                break;

            case typeB:
                produit = new ProduitB();
                break;

            case typeC:
                produit = new ProduitC();
                break;

            case typeD:
                produit = new ProduitD();
                break;

            default:
                throw new IllegalArgumentException(
                        "Type de produit inconnu"
                );
        }

        return produit;
    }

}
```

Listing 9 – Classe ProduitA

```
public class ProduitA extends Produit {

    public ProduitA() {

    }

    public void methodeproduit() {
        System.out.println("je suis un produit de type A");
        System.out.println("produitA.methodeproduit()");
    }

}
```

Listing 10 – Classe ProduitB

```
public class ProduitB extends Produit {

    public ProduitB() {

    }

    public void methodeproduit() {
        System.out.println("je suis un produit de type B");
        System.out.println("produitB.methodeproduit()");
    }

}
```

Listing 11 – Classe ProduitC

```

```

```java
public class ProduitC extends Produit {

    public ProduitC() {

    }

    public void methodeproduit() {
        System.out.println("je suis un produit de type C");
        System.out.println("produitC.methodeproduit()");
    }

}
```

Listing 12 – Classe ProduitD

```java

public class ProduitD extends Produit {

    public ProduitD() {

    }

    public void methodeproduit() {
        System.out.println("je suis un produit de type D");
        System.out.println("produitD.methodeproduit()");
    }

}
```

Listing 13 – Classe Client

```java

public class Client {

    public static void main(String[] args) {

        System.out.println("bonjour le monde");

        FabriqueProduit fabriqueproduit = new FabriqueProduit();

        Produit produit = null;

        // Affiche un produit de type A
        produit = fabriqueproduit.createProduit(fabriqueproduit.typeA);
        produit.methodeproduit();

        // Affiche un produit de type B
        produit = fabriqueproduit.createProduit(fabriqueproduit.typeB);
        produit.methodeproduit();

        // Affiche un produit de type C
        produit = fabriqueproduit.createProduit(fabriqueproduit.typeC);
        produit.methodeproduit();

         // Affiche un produit de type D
        produit = fabriqueproduit.createProduit(fabriqueproduit.typeD);
        produit.methodeproduit();

        // Produit inexistant
```

```
        String e = "E";

        produit = fabriqueproduit.createProduit(e);
        produit.methodeproduit();

    }

}
```

5 Diagrammes UML

5.1 1. Méthode Factory

Voici le diagramme UML initial de la factory simple :

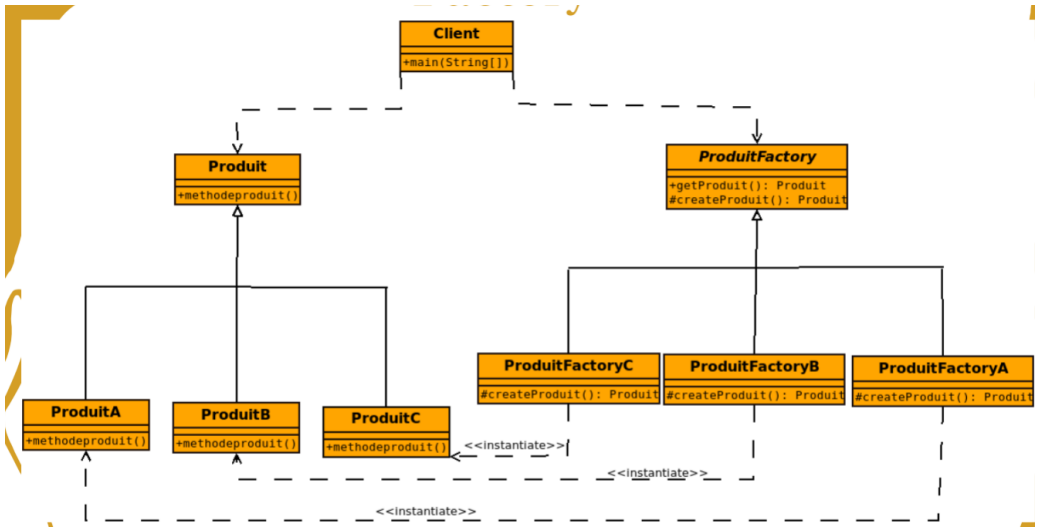


FIGURE 3 – Diagramme UML - Factory method

6 Implémentation Java

6.1 Capture de l'environnement de développement

L'image ci-dessous montre la structure du projet sous IntelliJ IDEA :

Listing 14 – Classe Produit

```
public abstract class Produit {

    public abstract void methodeproduit();

}
```

Listing 15 – Classe ProduitFactory

```
public abstract class ProduitFactory {

    public Produit getProduit() {
```

```
5
6           return createProduit();
7
8       }
9
10      protected abstract Produit createProduit();
11
12  }
```

Listing 16 – Classe ProduitFactoryA

```
1
2   public class ProduitFactoryA extends ProduitFactory {
3
4       protected Produit createProduit() {
5
6           Produit produit = null;
7
8           produit = new ProduitA();
9
10          return produit;
11
12      }
13
14  }
```

Listing 17 – Classe ProduitFactoryB

```
1
2   public class ProduitFactoryB extends ProduitFactory {
3
4       protected Produit createProduit() {
5
6           Produit produit = null;
7
8           produit = new ProduitB();
9
10          return produit;
11
12      }
13
14  }
```

Listing 18 – Classe ProduitFactoryC

```
1
2   public class ProduitFactoryC extends ProduitFactory {
3
4       protected Produit createProduit() {
5
6           Produit produit = null;
7
8           produit = new ProduitC();
9
10          return produit;
11
12      }
13
14  }
```

Listing 19 – Classe ProduitA

```
1
2 public class ProduitA extends Produit {
3
4     public void methodeproduit() {
5
6         System.out.println("je␣suis␣un␣produit␣de␣type␣A");
7         System.out.println("produitA.methodeproduit()");
8
9     }
10
11 }
```

Listing 20 – Classe ProduitB

```
1
2 public class ProduitB extends Produit {
3
4     public void methodeproduit() {
5
6         System.out.println("je␣suis␣un␣produit␣de␣type␣B");
7         System.out.println("produitB.methodeproduit()");
8
9     }
10
11 }
```

Listing 21 – Classe ProduitC

```
1
2 public class ProduitC extends Produit {
3
4     public void methodeproduit() {
5
6         System.out.println("je␣suis␣un␣produit␣de␣type␣C");
7         System.out.println("produitC.methodeproduit()");
8
9     }
10
11 }
```

Listing 22 – Classe Client

```
1
2 public class Client {
3
4     public static void main(String[] args) {
5
6         System.out.println("..........Methode␣2:␣Abstract␣Factory
              ..........");
7
8         ProduitFactory produitfactoryA = new ProduitFactoryA();
9
10        ProduitFactory produitfactoryB = new ProduitFactoryB();
11
12        ProduitFactory produitfactoryC = new ProduitFactoryC();
13
14        Produit produit = null;
15
```

```
        // Affiche un produit de type A
        produit = produitfactoryA.getProduit();
        produit.methodeproduit();

        // Affiche un produit de type B
        produit = produitfactoryB.getProduit();
        produit.methodeproduit();

        // Affiche un produit de type C
        produit = produitfactoryC.getProduit();
        produit.methodeproduit();

    }

}
```

6.2 2. Design Pattern Factory Modifié

Après restructuration, nous obtenons le diagramme UML suivant avec la méthode :

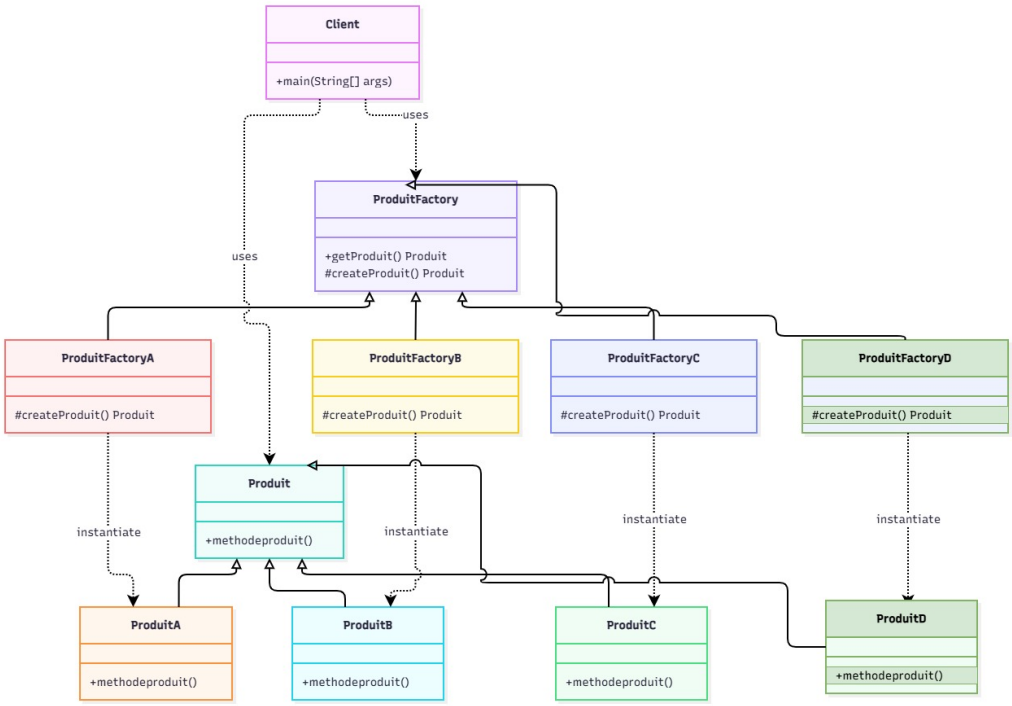


FIGURE 4 – Diagramme UML - Factory Modifié

Listing 23 – Classe Produit

```

public abstract class Produit {

    public abstract void methodeproduit();

}
```

Listing 24 – Classe ProduitFactory

```

```

```
2  public abstract class ProduitFactory {
3
4      public Produit getProduit() {
5
6          return createProduit();
7
8      }
9
10     protected abstract Produit createProduit();
11
12 }
```

Listing 25 – Classe ProduitFactoryA

```
1
2  public class ProduitFactoryA extends ProduitFactory {
3
4      protected Produit createProduit() {
5
6          Produit produit = null;
7
8          produit = new ProduitA();
9
10         return produit;
11
12     }
13
14 }
```

Listing 26 – Classe ProduitFactoryB

```
1
2  public class ProduitFactoryB extends ProduitFactory {
3
4      protected Produit createProduit() {
5
6          Produit produit = null;
7
8          produit = new ProduitB();
9
10         return produit;
11
12     }
13
14 }
```

Listing 27 – Classe ProduitFactoryC

```
1
2  public class ProduitFactoryC extends ProduitFactory {
3
4      protected Produit createProduit() {
5
6          Produit produit = null;
7
8          produit = new ProduitC();
9
10         return produit;
11
```

```
12      }
13
14  }
```

Listing 28 – Classe ProduitFactoryD

```
1
2   public class ProduitFactoryD extends ProduitFactory {
3
4       protected Produit createProduit() {
5
6           Produit produit = null;
7
8           produit = new ProduitD();
9
10          return produit;
11
12      }
13
14  }
```

Listing 29 – Classe ProduitA

```
1
2   public class ProduitA extends Produit {
3
4       public void methodeproduit() {
5
6           System.out.println("je␣suis␣un␣produit␣de␣type␣A");
7           System.out.println("produitA.methodeproduit()");
8
9       }
10
11  }
```

Listing 30 – Classe ProduitB

```
1
2   public class ProduitB extends Produit {
3
4       public void methodeproduit() {
5
6           System.out.println("je␣suis␣un␣produit␣de␣type␣B");
7           System.out.println("produitB.methodeproduit()");
8
9       }
10
11  }
```

Listing 31 – Classe ProduitC

```
1
2   public class ProduitC extends Produit {
3
4       public void methodeproduit() {
5
6           System.out.println("je␣suis␣un␣produit␣de␣type␣C");
7           System.out.println("produitC.methodeproduit()");
8
9       }
```

```
10
11 }
```

Listing 32 – Classe ProduitD

```
1
2 public class ProduitD extends Produit {
3
4     public void methodeproduit() {
5
6         System.out.println("je␣suis␣un␣produit␣de␣type␣D");
7         System.out.println("produitD.methodeproduit()");
8
9     }
10
11 }
```

Listing 33 – Classe Client

```
1
2 public class Client {
3
4     public static void main(String[] args) {
5
6         System.out.println("..........Methode␣2:␣Abstract␣Factory
              .........");
7
8         ProduitFactory produitfactoryA = new ProduitFactoryA();
9
10        ProduitFactory produitfactoryB = new ProduitFactoryB();
11
12        ProduitFactory produitfactoryC = new ProduitFactoryC();
13
14        ProduitFactory produitfactoryD = new ProduitFactoryD();
15
16        Produit produit = null;
17
18        // Affiche un produit de type A
19        produit = produitfactoryA.getProduit();
20        produit.methodeproduit();
21
22        // Affiche un produit de type B
23        produit = produitfactoryB.getProduit();
24        produit.methodeproduit();
25
26        // Affiche un produit de type C
27        produit = produitfactoryC.getProduit();
28        produit.methodeproduit();
29
30        // Affiche un produit de type D
31        produit = produitfactoryD.getProduit();
32        produit.methodeproduit();
33
34    }
35
36 }
```